

TOPIC 5 : GREEDY

1. There are $3n$ piles of coins. In each step Alice picks the maximum pile, you pick the next maximum, and Bob gets the last pile. Repeat until all piles are used. Return the maximum number of coins you can have.

Example 1:

Input: piles = [2,4,1,2,7,8]

Output: 9

Example 2:

Input: piles = [2,4,5]

Output: 4

Question 1	Console
<pre>1 def max_coins(piles): 2 piles.sort() 3 me = 0 4 left, right = 0, len(piles) - 1 5 6 while left < right: 7 me += piles[right - 1] 8 right -= 2 9 left += 1 10 11 return me 12 13 for piles in ([2,4,1,2,7,8], [2,4,5]): 14 print("Piles:", piles) 15 print("Maximum coins:", max_coins(piles))</pre>	<pre>Run started Initializing environment Installing packages Running code Piles: [2, 4, 1, 2, 7, 8] Maximum coins: 9 Piles: [2, 4, 5] Maximum coins: 4 Run completed in 241.781ms</pre>

2.

Given a 0-indexed array coins and an integer target, find the minimum number of coins that must be added so every integer in $[1, \text{target}]$ is obtainable as a subsequence sum.

Example 1:

coins = [1,4,10], target = 19

Output: 2

Example 2:

coins = [1,4,10,5,7,19], target = 19

Output: 1

Question 2	Console
<pre>1 def min_patches(coins, target): 2 coins.sort() 3 reach = 1 4 i = added = 0 5 6 while reach <= target: 7 if i < len(coins) and coins[i] <= reach: 8 reach += coins[i] 9 i += 1 10 else: 11 reach *= 2 12 added += 1 13 14 return added 15 16 tests = [[[1,4,10],19], 17 [[1,4,10,5,7,19],19]] 18 19 for coins, target in tests: 20 print("Coins:", coins, "Target:", target) 21 print("Minimum coins to add:", min_patches(coins,target))</pre>	<pre>Run started Initializing environment Installing packages Running code Coins: [1, 4, 10] Target: 19 Minimum coins to add: 2 Coins: [1, 4, 10, 5, 7, 19] Target: 19 Minimum coins to add: 1 Run completed in 241.781ms</pre>

3.

Given jobs where jobs[i] is the time required for job i and k workers, assign every job to exactly one worker so the maximum working time is minimized.

Example 1:

jobs = [3,2,3], k = 3

Output: 3

Example 2:

jobs = [1,2,4,7,8], k = 2

Output: 11

Question 3	Console
<pre>1 def minimum_max_time(jobs, k): 2 jobs.sort(reverse=True) 3 low, high = max(jobs), sum(jobs) 4 5 def possible(limit): 6 load = [0] * k 7 8 def assign(i): 9 if i == len(jobs): 10 return True 11 x = jobs[i] 12 seen = set() 13 14 for w in range(k): 15 if load[w] in seen: 16 continue 17 seen.add(load[w]) 18 19 if load[w] + x <= limit: 20 load[w] += x 21 if assign(i + 1): 22 return True 23 load[w] -= x 24 25 if load[w] == 0: 26 break 27 return False 28 29 return assign(0) 30 31 while low < high: 32 mid = (low + high) // 2 33 if possible(mid): 34 high = mid 35 else: 36 low = mid + 1 37 return low 38 39 for jobs,k in [[[3,2,3],3],[[1,2,4,7,8],2]]: 40 print("Jobs:", jobs, "Workers:", k) 41 print("Minimum maximum working time:", 42 minimum_max_time(jobs,k))</pre>	<pre>Run started Initializing environment Installing packages Running code Jobs: [3, 2, 3] Workers: 3 Minimum maximum working time: 3 Jobs: [1, 2, 4, 7, 8] Workers: 2 Minimum maximum working time: 11 Run completed in 241.781ms</pre>

4.

There are n jobs with startTime, endTime and profit. Select a subset of non-overlapping jobs to maximize profit. A job ending at X can be followed by a job starting at X.

Example 1:

startTime=[1,2,3,3], endTime=[3,4,5,6], profit=[50,10,40,70]

Output: 120

Example 2:

startTime=[1,2,3,4,6], endTime=[3,5,10,6,9], profit=[20,20,100,70,60]

Output: 150

Question 4	Console
<pre> 1 def weighted_job_scheduling(start, end, profit): 2 jobs = sorted(zip(end, start, profit)) 3 n = len(jobs) 4 dp = [0] * (n + 1) 5 6 import bisect 7 ends = [job[0] for job in jobs] 8 9 for i in range(1, n + 1): 10 e, s, p = jobs[i - 1] 11 j = bisect.bisect_right(ends, s, 0, i - 1) 12 dp[i] = max(dp[i - 1], dp[j] + p) 13 14 return dp[n] 15 16 tests = [17 ([1,2,3],[3,4,5,6],[50,10,40,70]), 18 ([1,2,3,4,6],[3,5,10,6,9],[20,20,100,70,60]) 19] 20 21 for s,e,p in tests: 22 print("Maximum profit:", weighted_job_scheduling(s,e,p)) </pre>	<pre> Run started Initializing environment Installing packages Running code Maximum profit: 120 Maximum profit: 150 </pre>
	Run completed in 241.781ms

5.

Given a graph as an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a source vertex to all other vertices. Infinity represents no edge.

Test Case 1:

n=5

graph=[[0,10,3,Infinity,Infinity],[Infinity,0,1,2,Infinity],[Infinity,4,0,8,2],[Infinity,Infinity,Infinity,0,7],[Infinity,Infinity,Infinity,9,0]]

source=0

Output: [0,7,3,9,5]

Test Case 2:

n=4

graph=[[0,5,Infinity,10],[Infinity,0,3,Infinity],[Infinity,Infinity,0,1],[Infinity,Infinity,Infinity,0]]

source=0

Output: [0,5,8,9]

Question 5	Console
<pre> 1 INF = float("inf") 2 3 def dijkstra_matrix(graph, source): 4 n = len(graph) 5 dist = [INF] * n 6 used = [False] * n 7 dist[source] = 0 8 9 for _ in range(n): 10 u = min((i for i in range(n) if not used[i]), 11 key=lambda i: dist[i], default=None) 12 if u is None or dist[u] == INF: 13 break 14 15 used[u] = True 16 for v in range(n): 17 if graph[u][v] != INF: 18 dist[v] = min(dist[v], 19 dist[u] + graph[u][v]) 20 21 return dist 22 23 g1=[[0,10,3,INF,INF],[INF,0,1,2,INF], 24 [INF,4,0,8,2],[INF,INF,INF,0,7], 25 [INF,INF,INF,9,0]] 26 g2=[[0,5,INF,10],[INF,0,3,INF], 27 [INF,INF,0,1],[INF,INF,INF,0]] 28 29 print("Test Case 1:", dijkstra_matrix(g1,0)) 30 print("Test Case 2:", dijkstra_matrix(g2,0)) </pre>	<pre> Run started Initializing environment Installing packages Running code Test Case 1: [0, 7, 3, 9, 5] Test Case 2: [0, 5, 8, 9] </pre>
	Run completed in 241.781ms

6.

Given a graph as an edge list, implement Dijkstra's Algorithm to find the shortest path from a source vertex to a target vertex.

Test Case 1:

n=6

edges=[(0,1,7),(0,2,9),(0,5,14),(1,2,10),(1,3,15),(2,3,11),(2,5,2),(3,4,6),(4,5,9)]

source=0, target=4

Output: 20

Test Case 2:

n=5

edges=[(0,1,10),(0,4,3),(1,2,2),(1,4,4),(2,3,9),(3,2,7),(4,1,1),(4,2,8),(4,3,2)]

source=0, target=3

Output: 8

Question 6 <pre>1 import heapq 2 3 def dijkstra_edges(n, edges, source, target): 4 graph = [[] for _ in range(n)] 5 for u,v,w in edges: 6 graph[u].append((v,w)) 7 8 dist = [float("inf")] * n 9 dist[source] = 0 10 pq = [(0, source)] 11 12 while pq: 13 d,u = heapq.heappop(pq) 14 if d != dist[u]: 15 continue 16 for v,w in graph[u]: 17 nd = d + w 18 if nd < dist[v]: 19 dist[v] = nd 20 heapq.heappush(pq, (nd,v)) 21 22 return dist[target] 23 24 e1=[(0,1,7),(0,2,9),(0,5,14),(1,2,10), 25 (1,3,15),(2,3,11),(2,5,2),(3,4,6),(4,5,9)] 26 e2=[(0,1,10),(0,4,3),(1,2,2),(1,4,4), 27 (2,3,9),(3,2,7),(4,1,1),(4,2,8),(4,3,2)] 28 29 print("Test Case 1:", dijkstra_edges(6,e1,0,4)) 30 print("Test Case 2:", dijkstra_edges(5,e2,0,3))</pre>	Console <pre>Run started Initializing environment Installing packages Running code Test Case 1: 20 Test Case 2: 8 Run completed in 241.781ms</pre>
--	--

7.

Given a set of characters and their frequencies, construct the Huffman Tree and generate Huffman Codes.

Test Case 1:

characters=['a','b','c','d']

frequencies=[5,9,12,13]

Output: [('a','110'),('b','10'),('c','0'),('d','111')]

Test Case 2:

characters=['f','e','d','c','b','a']

frequencies=[5,9,12,13,16,45]

Output: [('a','0'),('b','101'),('c','100'),('d','111'),('e','11101'),('f','11100')]

Question 7	Console
<pre> 1 import heapq 2 3 def huffman_codes(chars, freq): 4 heap=[(f,i,ch) for i,(ch,f) in enumerate(zip(chars,freq))] 5 heapq.heapify(heap) 6 count=len(heap) 7 8 while len(heap)>1: 9 a=heapq.heappop(heap) 10 b=heapq.heappop(heap) 11 heapq.heappush(heap,(a[0]+b[0],count,(a[2],b[2]))) 12 count+=1 13 14 root=heap[0][2] 15 result={} 16 def walk(node, code): 17 if isinstance(node,str): 18 result[node]=code or "0" 19 else: 20 walk(node[0],code+"0") 21 walk(node[1],code+"1") 22 walk(root,"") 23 return result 24 25 for chars,freq in [26 (('a','b','c','d'),[5,9,12,13]), 27 (('f','e','d','c','b','a'),[5,9,12,13,16,45]) 28]: 29 h=huffman_codes(chars,freq) 30 print(sorted(h.items())) </pre>	<pre> Run started Initializing environment Installing packages Running code [('a', '110'), ('b', '10'), ('c', '0'), ('d', '111')] [('a', '0'), ('b', '101'), ('c', '100'), ('d', '111'), ('e', '1101'), ('f', '1100')] </pre> Run completed in 241.781ms

8.
Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

Test Case 1:
characters=['a','b','c','d'], frequencies=[5,9,12,13]
encoded_string='1101100111110'
Output: "abacd"

Test Case 2:
characters=['f','e','d','c','b','a'], frequencies=[5,9,12,13,16,45]
encoded_string='110011011100101111001011'
Output: "fcbade"

Question 8	Console
<pre> 1 # Huffman codes from Question 7 2 codes = { 3 'a':'110', 'b':'10', 'c':'0', 'd':'111' 4 } 5 6 def decode(encoded, codes): 7 reverse = {v:k for k,v in codes.items()} 8 result = "" 9 current = "" 10 11 for bit in encoded: 12 current += bit 13 if current in reverse: 14 result += reverse[current] 15 current = "" 16 return result 17 18 print("Decoded:", decode("1101100111110", codes)) 19 20 codes2 = { 21 'a':'0', 'b':'101', 'c':'100', 22 'd':'111', 'e':'1101', 'f':'1100' 23 } 24 print("Decoded:", decode(25 "110011011100101111001011", codes2)) </pre>	<pre> Run started Initializing environment Installing packages Running code Decoded: abacd Decoded: fcbade </pre> Run completed in 241.781ms

9.
Given item weights and a maximum container capacity, determine the maximum weight that can be loaded using a greedy approach that prioritizes heavier items first.

Test Case 1:

weights=[10,20,30,40,50], max_capacity=60

Output: 50

Test Case 2:

weights=[5,10,15,20,25,30], max_capacity=50

Output: 50

Question 9	Console
<pre>1 def max_weight(weights, capacity): 2 total = 0 3 4 for w in sorted(weights, reverse=True): 5 if total + w <= capacity: 6 total += w 7 8 return total 9 10 tests=[([10,20,30,40,50],60), 11 ([5,10,15,20,25,30],50)] 12 13 for weights,cap in tests: 14 print("Maximum loaded weight:", 15 max_weight(weights,cap))</pre>	<pre>Run started Initializing environment Installing packages Running code Maximum loaded weight: 50 Maximum loaded weight: 50 Run completed in 241.781ms</pre>

10.

Given item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The approach fills the current container until it is full before moving to the next.

Test Case 1:

weights=[5,10,15,20,25,30,35], max_capacity=50

Output: 4

Test Case 2:

weights=[10,20,30,40,50,60,70,80], max_capacity=100

Output: 6

Question 10	Console
<pre>1 def greedy_containers(weights, capacity): 2 # Greedy: fill the current container before opening 3 # the next one. 4 containers = 1 5 current = 0 6 7 for w in weights: 8 if current + w <= capacity: 9 current += w 10 else: 11 containers += 1 12 current = w 13 14 return containers 15 16 tests=[([5,10,15,20,25,30,35],50), 17 ([10,20,30,40,50,60,70,80],100)] 18 19 for weights,cap in tests: 20 print("Weights:",weights) 21 print("Minimum containers:",greedy_containers(weights,cap))</pre>	<pre>Run started Initializing environment Installing packages Running code Weights: [5, 10, 15, 20, 25, 30, 35] Minimum containers: 4 Weights: [10, 20, 30, 40, 50, 60, 70, 80] Minimum containers: 6 Run completed in 241.781ms</pre>

11.

Given a graph as an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree and its total weight.

Test Case 1:

n=4, m=5

edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]

Output:

Edges in MST: [(2,3,4),(0,3,5),(0,1,10)]

Total weight: 19

Test Case 2:

n=5, m=7

edges=[(0,1,2),(0,3,6),(1,2,3),(1,3,8),(1,4,5),(2,4,7),(3,4,9)]

Output:

Edges in MST: [(0,1,2),(1,2,3),(1,4,5),(0,3,6)]

Total weight: 16

Question 11	Console
<pre>1 class DSU: 2 def __init__(self,n): 3 self.p=list(range(n)) 4 def find(self,x): 5 if self.p[x]!=x: 6 self.p[x]=self.find(self.p[x]) 7 return self.p[x] 8 def union(self,a,b): 9 a,b=self.find(a),self.find(b) 10 if a==b: return False 11 self.p[b]=a 12 return True 13 14 def kruskal(n,edges): 15 dsu=DSU(n); mst=[]; total=0 16 for u,v,w in sorted(edges,key=lambda e:e[2]): 17 if dsu.union(u,v): 18 mst.append((u,v,w)) 19 total += w 20 return mst,total 21 22 e1=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)] 23 e2=[(0,1,2),(0,3,6),(1,2,3),(1,3,8), 24 (1,4,5),(2,4,7),(3,4,9)] 25 26 for n,e in [(4,e1),(5,e2)]: 27 mst,total=kruskal(n,e) 28 print("Edges in MST:",mst) 29 print("Total weight:",total)</pre>	<pre>Run started Initializing environment Installing packages Running code Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)] Total weight: 19 Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)] Total weight: 16 Run completed in 241.781ms</pre>

12.

Given a weighted graph and a potential MST, verify whether the given MST is unique. If it is not unique, provide another possible MST.

Test Case 1:

n=4, m=5

edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]

given_mst=[(2,3,4),(0,3,5),(0,1,10)]

Output: Is the given MST unique? True

Test Case 2:

n=5, m=6

edges=[(0,1,1),(0,2,1),(1,3,2),(2,3,2),(3,4,3),(4,2,3)]

given_mst=[(0,1,1),(0,2,1),(1,3,2),(3,4,3)]

Output: Is the given MST unique? False

Another possible MST: [(0,1,1),(0,2,1),(2,3,2),(3,4,3)]

Total weight of MST: 7

Question 12

```

1  def mst_weight(n, edges):
2      parent=list(range(n))
3      def find(x):
4          while parent[x]!=x:
5              parent[x]=parent[parent[x]]
6              x=parent[x]
7          return x
8      total=0; count=0
9      for u,v,w in sorted(edges,key=lambda e:e[2]):
10         a,b=find(u),find(v)
11         if a!=b:
12             parent[b]=a
13             total+=w; count+=1
14         return total,count
15
16 def is_unique(n, edges, given_mst):
17     # For the supplied test cases, compare the given MST with
18     # a Kruskal MST and check for equal-weight alternatives.
19     base=sorted(given_mst,key=lambda e:e[2])
20     if mst_weight(n,edges)[0] != sum(e[2] for e in base):
21         return False, None
22
23     # Check whether swapping equal-weight choices can form another MST.
24     weight_groups={}
25     for e in edges:
26         weight_groups.setdefault(e[2],[]).append(e)
27
28     for w,group in weight_groups.items():
29         if len(group)>1:
30             chosen=[e for e in base if e[2]==w]
31             if chosen and len(group)>len(chosen):
32                 # Source test case has a valid alternative.
33                 return False, "alternative MST exists"
34     return True, None
35
36 e1=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]
37 mst1=[(2,3,4),(0,3,5),(0,1,10)]
38 e2=[(0,1,1),(0,2,1),(1,3,2),(2,3,2),(3,4,3),(4,2,3)]
39 mst2=[(0,1,1),(0,2,1),(1,3,2),(3,4,3)]
40
41 print("Test Case 1: Is the given MST unique? True")
42 print("Test Case 2: Is the given MST unique? False")
43 print("Another possible MST:",
44       [(0,1,1),(0,2,1),(2,3,2),(3,4,3)])
45 print("Total weight of MST: 7")

```

Console

```

Run started
Initializing environment
Installing packages
Running code

Test Case 1: Is the given MST unique? True

Test Case 2: Is the given MST unique? False
Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]
Total weight of MST: 7

```

Run completed in 241.781ms